

Python Interview Cheat Sheet

For ML Engineer / AI Developer Roles

Lists • Strings • Dictionaries • Numbers • Sets • Matrix

This guide covers the most commonly tested Python patterns in technical interviews. Each section includes ready-to-use code, tips for interview situations, and a quick scenario reference table. Practice each pattern until your hands write the code automatically.

Section	Topics Covered
1. Lists	Merge, Sort, Filter, Deduplicate, Rotate, Slice, Two Sum
2. Strings	Reverse, Palindrome, Anagram, Frequency, Vowels, Formatting
3. Dictionaries	Frequency count, Sort by value, Merge, Invert, Group
4. Numbers	Prime, Fibonacci, Factorial, GCD, Power, Digit Sum
5. Sets	Union, Intersection, Difference, Subset, Deduplication
6. Matrix / 2D Lists	Access, Transpose, Flatten, Row sums, Diagonal
7. Scenario Map	Quick reference: question → pattern → code
8. Interview Rules	3 rules to never freeze under pressure again

1. LISTS

■ Lists are the most tested data structure. Master + (merge), sorted(), set(), and list comprehension.

Merge & Sort

```
a = [1, 3, 2]
b = [6, 4, 5]

# Merge using +
merged = a + b # [1, 3, 2, 6, 4, 5]

# Merge using extend (modifies a in place)
a.extend(b) # a is now [1, 3, 2, 6, 4, 5]

# Sort (returns new list)
sorted(merged) # [1, 2, 3, 4, 5, 6]
sorted(merged, reverse=True) # [6, 5, 4, 3, 2, 1]

# Sort in place (modifies original)
merged.sort() # modifies merged directly

# One liner: merge + sort
result = sorted(a + b) # [1, 2, 3, 4, 5, 6]
```

Deduplicate

```
a = [1, 1, 2, 3, 3, 4]

# Remove duplicates
list(set(a)) # [1, 2, 3, 4] (order not guaranteed)
sorted(set(a)) # [1, 2, 3, 4] (sorted)

# Remove duplicates keeping order (without set)
result = []
for item in a:
    if item not in result:
        result.append(item)
```

Filter & Search

```
a = [1, 2, 3, 4, 5, 6]

# Filter using list comprehension
[x for x in a if x > 3] # [4, 5, 6]
[x for x in a if x % 2 == 0] # [2, 4, 6] (even numbers)
[x for x in a if x % 2 != 0] # [1, 3, 5] (odd numbers)

# Find max/min without built-ins
max_val = a[0]
for x in a:
    if x > max_val:
        max_val = x # max_val = 6

# Second largest
sorted(set(a))[-2] # 5
```

Set Operations on Lists

```
a = [1, 2, 3, 4]
```

```

b = [3, 4, 5, 6]

# Common elements (intersection)
[x for x in a if x in b] # [3, 4]
list(set(a) & set(b)) # [3, 4]

# Elements only in a (difference)
list(set(a) - set(b)) # [1, 2]

# Elements in either but not both
list(set(a) ^ set(b)) # [1, 2, 5, 6]

# All unique elements (union)
sorted(set(a) | set(b)) # [1, 2, 3, 4, 5, 6]

```

Slicing, Rotating & Chunking

```

a = [1, 2, 3, 4, 5]

# Slice
a[1:4] # [2, 3, 4]
a[::-1] # [5, 4, 3, 2, 1] (reverse)
a[::2] # [1, 3, 5] (every 2nd element)

# Rotate by k positions
k = 2
rotated = a[k:] + a[:k] # [3, 4, 5, 1, 2]

# Chunk into groups of n
n = 2
[a[i:i+n] for i in range(0, len(a), n)] # [[1,2],[3,4],[5]]

# Flatten nested list
nested = [[1,2],[3,4],[5,6]]
[x for sub in nested for x in sub] # [1, 2, 3, 4, 5, 6]

```

Two Sum — Very Common

```

def two_sum(arr, target):
    seen = {} # store number: index
    for i, num in enumerate(arr):
        diff = target - num
        if diff in seen:
            return [seen[diff], i]
        seen[num] = i
    two_sum([2, 7, 11, 15], 9) # [0, 1]

```

Sort Without Built-ins

```

# Bubble Sort – always acceptable in interviews
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j] # swap
    return arr

# Selection Sort
def selection_sort(arr):

```

```
for i in range(len(arr)):
    min_idx = i
    for j in range(i+1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
return arr
```

■ Interview trick: When they say 'without built-ins', they mean without `sorted()` or `.sort()`. Using `a + b` to merge is always fine.

■ 2. STRINGS

■ Strings are immutable in Python. Most operations return a new string. Memorise `[::-1]` for reverse.

Reverse & Palindrome

```
s = 'hello'

# Reverse a string
s[::-1] # 'olleh'

# Reverse manually (if asked without slicing)
result = ''
for ch in s:
    result = ch + result # 'olleh'

# Palindrome check
def is_palindrome(s):
    s = s.lower().replace(' ', '')
    return s == s[::-1]

is_palindrome('racecar') # True
is_palindrome('A man a plan a canal Panama') # True
```

Frequency & Count

```
s = 'hello world'

# Count specific character
s.count('l') # 3

# Frequency of every character
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
# {'h':1, 'e':1, 'l':3, 'o':2, ' ':1, 'w':1, 'r':1, 'd':1}

# Most frequent character
max(freq, key=freq.get) # 'l'

# Count vowels
sum(1 for ch in s if ch in 'aeiouAEIOU') # 3
```

Anagram & Comparison

```
# Anagram check
sorted('listen') == sorted('silent') # True
sorted('hello') == sorted('world') # False

# Case-insensitive anagram
def is_anagram(s1, s2):
    return sorted(s1.lower()) == sorted(s2.lower())
```

String Formatting & Manipulation

```
s = 'hello world'

s.upper() # 'HELLO WORLD'
s.lower() # 'hello world'
s.title() # 'Hello World'
```

```
s.strip() # remove leading/trailing spaces
s.replace(' ', '_') # 'hello_world'
s.replace('l', 'x') # 'hexxo worxd'
s.split() # ['hello', 'world']
s.split('o') # ['hell', ' w', 'rld']
' '.join(['hello', 'world']) # 'hello world'
s.startswith('he') # True
s.endswith('ld') # True
'123'.isdigit() # True
'12a'.isdigit() # False
s.find('world') # 6 (index)
'world' in s # True
```

Word Operations

```
sentence = 'the cat sat on the mat'

# Word frequency
freq = {}
for word in sentence.split():
    freq[word] = freq.get(word, 0) + 1
# {'the': 2, 'cat': 1, 'sat': 1, 'on': 1, 'mat': 1}

# Reverse each word
' '.join(w[::-1] for w in sentence.split())
# 'eht tac tas no eht tam'

# Reverse word order
' '.join(sentence.split()[::-1])
# 'mat the on sat cat the'
```

■ 3. DICTIONARIES

■ Dictionaries are key for frequency counting. Always use `.get(key, 0)` instead of direct access to avoid `KeyError`.

Basic Operations

```
d = {'a': 1, 'b': 2, 'c': 3}

# Access safely (no KeyError)
d.get('z', 0) # 0 (default if missing)
d.get('a', 0) # 1

# Check if key exists
'a' in d # True
'z' in d # False

# Loop through
for key in d: print(key) # keys
for val in d.values(): print(val) # values
for k, v in d.items(): print(k, v) # both

# Get all keys / values as list
list(d.keys()) # ['a', 'b', 'c']
list(d.values()) # [1, 2, 3]
```

Sort, Merge & Invert

```
d = {'b': 3, 'a': 1, 'c': 2}

# Sort by value (ascending)
sorted(d.items(), key=lambda x: x[1])
# [('a', 1), ('c', 2), ('b', 3)]

# Sort by key
sorted(d.items(), key=lambda x: x[0])
# [('a', 1), ('b', 3), ('c', 2)]

# Merge two dicts
d1 = {'a': 1}; d2 = {'b': 2}
merged = {**d1, **d2} # {'a': 1, 'b': 2}

# Invert (swap keys and values)
inverted = {v: k for k, v in d.items()}
# {3: 'b', 1: 'a', 2: 'c'}
```

Frequency Counting — Most Common Pattern

```
# Character frequency in string
def char_freq(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    return freq

# Word frequency in sentence
def word_freq(sentence):
    freq = {}
```

```
for word in sentence.split():
    freq[word] = freq.get(word, 0) + 1
return freq

# Most frequent element in list
def most_frequent(arr):
    freq = {}
    for item in arr:
        freq[item] = freq.get(item, 0) + 1
    return max(freq, key=freq.get)
```

Group By Condition

```
nums = [1, 2, 3, 4, 5, 6]

# Group even and odd
grouped = {'even': [], 'odd': []}
for n in nums:
    grouped['even' if n % 2 == 0 else 'odd'].append(n)
# {'even': [2,4,6], 'odd': [1,3,5]}

# Group strings by first letter
words = ['apple', 'ant', 'banana', 'bat', 'cherry']
groups = {}
for w in words:
    key = w[0]
    groups.setdefault(key, []).append(w)
# {'a': ['apple', 'ant'], 'b': ['banana', 'bat'], 'c': ['cherry']}
```


■ 4. NUMBERS & MATH

■ For prime check, only loop up to $\sqrt{n}$. This is the key optimization interviewers look for.

Prime, Even/Odd, Divisibility

```
# Even / Odd
n % 2 == 0 # True if even
n % 2 != 0 # True if odd

# Divisibility
n % 3 == 0 # divisible by 3

# Prime check (optimised)
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5) + 1): # only up to sqrt(n)
        if n % i == 0:
            return False
    return True

# All primes up to N (Sieve)
def primes_up_to(n):
    return [x for x in range(2, n+1) if is_prime(x)]
```

Fibonacci & Factorial

```
# Fibonacci – iterative (preferred)
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
# fibonacci(7) = 13

# Fibonacci sequence as list
def fib_list(n):
    a, b, result = 0, 1, []
    for _ in range(n):
        result.append(a)
        a, b = b, a + b
    return result
# fib_list(7) = [0, 1, 1, 2, 3, 5, 8]

# Factorial – iterative
def factorial(n):
    result = 1
    for i in range(1, n+1):
        result *= i
    return result
# factorial(5) = 120
```

Number Manipulation

```
n = 12345
```

```
# Reverse a number
int(str(abs(n))[::-1]) # 54321

# Sum of digits
sum(int(d) for d in str(n)) # 15

# Count digits
len(str(abs(n))) # 5

# Power without built-in
def power(base, exp):
    result = 1
    for _ in range(exp):
        result *= base
    return result
# power(2, 8) = 256

# GCD (Greatest Common Divisor)
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
# gcd(48, 18) = 6

# LCM (Least Common Multiple)
def lcm(a, b):
    return a * b // gcd(a, b)
# lcm(4, 6) = 12
```

■ 5. SETS

■ Sets are unordered and contain only unique values. Use them for fast lookup and deduplication.

Core Set Operations

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# Union – all elements from both
a | b # {1, 2, 3, 4, 5, 6}
a.union(b) # same

# Intersection – only common
a & b # {3, 4}
a.intersection(b) # same

# Difference – in a but not b
a - b # {1, 2}
a.difference(b) # same

# Symmetric difference – not in both
a ^ b # {1, 2, 5, 6}
a.symmetric_difference(b) # same

# Subset / superset check
{1, 2}.issubset(a) # True ({1,2} inside a)
a.issuperset({1, 2}) # True

# Add / remove
a.add(10) # adds 10 to set
a.discard(10) # removes 10 (no error if missing)
a.remove(10) # removes 10 (KeyError if missing)
```

Practical Use Cases

```
# Remove duplicates from list
lst = [1, 1, 2, 3, 3, 4]
list(set(lst)) # [1, 2, 3, 4]
sorted(set(lst)) # [1, 2, 3, 4] sorted

# Fast membership check (faster than list)
valid = {1, 2, 3, 4, 5}
3 in valid # True (O(1) lookup)

# Find duplicates in a list
lst = [1, 2, 2, 3, 3, 3]
seen = set()
duplicates = set()
for x in lst:
    if x in seen:
        duplicates.add(x)
    seen.add(x)
# duplicates = {2, 3}
```

■ 6. MATRIX / 2D LISTS

■ Matrix questions are common in ML interviews. Transpose and flatten are the two most asked operations.

Access & Loop

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

# Access element [row][col]
matrix[0][0] # 1 (top-left)
matrix[1][2] # 6
matrix[2][2] # 9 (bottom-right)

# Loop row by row
for row in matrix:
    print(row) # [1,2,3] then [4,5,6] then [7,8,9]

# Loop element by element
for i in range(len(matrix)):
    for j in range(len(matrix[0])):
        print(matrix[i][j])

# Number of rows and columns
rows = len(matrix) # 3
cols = len(matrix[0]) # 3
```

Transpose, Flatten & Diagonal

```
# Transpose (rows become columns)
transposed = [[matrix[j][i] for j in range(len(matrix))]
              for i in range(len(matrix[0]))]
# [[1,4,7],[2,5,8],[3,6,9]]

# Flatten (2D to 1D)
[x for row in matrix for x in row] # [1,2,3,4,5,6,7,8,9]

# Row sums
[sum(row) for row in matrix] # [6, 15, 24]

# Column sums
[sum(matrix[r][c] for r in range(len(matrix)))
 for c in range(len(matrix[0]))] # [12, 15, 18]

# Main diagonal (top-left to bottom-right)
[matrix[i][i] for i in range(len(matrix))] # [1, 5, 9]
```

Create & Search

```
# Create N x M matrix filled with 0
n, m = 3, 4
grid = [[0] * m for _ in range(n)] # [[0,0,0,0],[0,0,0,0],[0,0,0,0]]

# Search for a value
def find_in_matrix(matrix, target):
    for i, row in enumerate(matrix):
        for j, val in enumerate(row):
```

```
if val == target:  
    return (i, j) # row, column  
return None
```

■ 7. SCENARIO QUICK REFERENCE

When the interviewer gives you a problem, match it to one of these patterns instantly:

Interview Says...	You Think...	Code
merge two lists	list + list	<code>sorted(a + b)</code>
remove duplicates	set()	<code>sorted(set(a))</code>
find common elements	intersection	<code>list(set(a) & set(b))</code>
elements in a not b	difference	<code>list(set(a) - set(b))</code>
count frequency	dict + .get()	<code>freq[x] = freq.get(x,0) + 1</code>
most frequent element	max with key	<code>max(freq, key=freq.get)</code>
sort without built-ins	bubble sort	nested loop + swap
check palindrome	reverse compare	<code>s == s[::-1]</code>
check anagram	sorted compare	<code>sorted(s1) == sorted(s2)</code>
find second largest	sort + index	<code>sorted(set(a))[-2]</code>
reverse string/list	slicing	<code>s[::-1]</code>
flatten nested list	comprehension	<code>[x for sub in lst for x in sub]</code>
transpose matrix	nested comprehension	<code>[[m[j][i] for j in ...]...]</code>
group even/odd	dict + loop	<code>d['even' if n%2==0 else 'odd']</code>
two numbers summing to target	two sum / hashmap	<code>seen = {}; diff = target - num</code>
chunk list into groups	range + slice	<code>[a[i:i+n] for i in range(0, len, n)]</code>
check prime	loop to sqrt	<code>range(2, int(n**0.5)+1)</code>
rotate list by k	slice + join	<code>a[k:] + a[:k]</code>
word frequency	split + dict	<code>freq[w] = freq.get(w,0) + 1</code>
invert a dictionary	dict comprehension	<code>{v: k for k, v in d.items()}</code>

■ 8. THE 3 RULES TO NEVER FREEZE

RULE 1: Talk First, Code Second

Before touching the keyboard, say your approach out loud: 'I'll merge the two lists with +, then sort using bubble sort.' This buys you thinking time and shows the interviewer you understand the problem. Silence + blank screen = panic. Talking + planning = confidence.

RULE 2: Write the Skeleton First

Never start with blank code. Write this first:

```
def solve(arr):  
    # edge case  
    # main logic  
    # return  
    pass
```

Fill in comments before actual code. You will never freeze on a blank screen again because you already have structure.

RULE 3: Test With a Tiny Example

After writing your solution, always say: 'Let me trace through with a = [1,3,2], b = [6,4,5]...' This shows confidence, catches bugs before the interviewer does, and buys you extra time to think. It also signals that you care about correctness — which is exactly what interviewers want to see.

Your Decomposition Framework

Every interview problem has at most 2-3 steps. Train yourself to split before you solve:

```
# Example: 'merge two unsorted lists into one sorted list'  
# Step 1: What do I have? → two lists  
# Step 2: What do I need? → one sorted list  
# Step 3: How? → merge with +, sort with sorted()  
result = sorted(a + b) # done  
  
# Example: 'find most common word in a sentence'  
# Step 1: What do I have? → a string  
# Step 2: What do I need? → most frequent word  
# Step 3: How? → split → count → max  
freq = {}  
for word in sentence.split():  
    freq[word] = freq.get(word, 0) + 1
```

```
result = max(freq, key=freq.get)
```

■ The problem always feels hard when you look at it as one big thing. Split it into steps. Each step is always simple.